

MiniJava 타입 시스템

Jens Palsberg 2014년 10

월

1 미니자바란 무엇인가?

미니자바는 자바의 하위 집합입니다. 미니자바 프로그램의 의미는 자바 프로그램으로서의 의미에 의해 결정됩니다. 미니자바에서는 오버로딩이 허용되지 않습니다. 미니자바 문 `System.out.println(...);`은 정수만 출력할 수 있습니다. 미니자바 식 `e.length`는 `int[]`형식의 식에만 적용됩니다.

이제 MiniJava의 구문과 타입 시스템을 정의하겠습니다. MiniJava 프로그램은 (아래에 명시된) MiniJava 타입 시스템에 따라 타입 검사가 통과될 때, 그리고 오직 그때만 (Java 언어 사양에 명시된) Java 타입 시스템에 따라 타입 검사가 통과됩니다.

2 구문

아래 문법은 다음과 같은 메타표기법을 사용합니다:

- 비단말 기호는 *o/ 글꼴*로 표기된 단어입니다.
- 종단 기호는 $\langle \text{IDENTIFIER} \rangle$ 와 $\langle \text{INTEGER LITERAL} \rangle$ 을 제외하고 이 글꼴로 표기됩니다.
- 규칙은 $lhs ::= rhs$ 의 형태를 띠며, 여기서 lhs 는 비종결 기호이고 rhs 는 비종결 기호와 종결 기호의 순서열로, 선택지는 $|$ 로 구분되며, 때로는 “...”를 사용하여 비어 있을 수도 있는 목록을 나타냅니다.
- 메타변수를 구별하기 위해 위첨자와 아래첨자를 사용할 것이다.

(목표) $g ::= mc\ d_1 \dots d_n$

(MainClass) $mc\ d_{(1)} \dots d_{(n)} ::= \text{class } id\ \{ \text{public static void main } (String\ []\ id^{SI})\{ t_1\ id_1; \dots; t_r\ id_r; s_1 \dots s_q \} \}$

(형식 선언) $d ::= \text{class } id\ \{ t_1\ id_1; \dots; t_f\ id_f; m_1 \dots m_k \}$

(메서드 선언) $m ::= \text{public } t\ id^M\ (t^F\ id^F_1, \dots, t^F\ id^F_n)\ \{ t_1\ id_1; \dots; t_r\ id_r; s_1 \dots s_q\ \text{return } e; \}$

(Type) $t ::= \text{int}[] \mid \text{boolean} \mid \text{int} \mid id$

(문) $s ::= \text{int}[] \mid \text{boolean} \mid \text{int} \mid id \mid id = e; \mid id[e(1)] = e_2; \mid \text{if } (e)\ s_1\ \text{else } s_2 \mid \text{while } (e)\ s \mid \text{System.out.println}(e);$

(식) $e ::= p_1 \ \&\& \ p_2 \mid p_1 < p_2 \mid p_1 + p_2 \mid p_1 - p_2 \mid p_1 * p_2 \mid p_1 [p_2] \mid p.length \mid p.id(e_1, \dots, e_n) \mid p$

(PrimaryExpression) $p ::= c \mid \text{true} \mid \text{false} \mid id \mid \text{this} \mid \text{new int}[e] \mid \text{new id}() \mid !e \mid (e)$

(정수 리터럴) $c ::= \langle \text{INTEGER LITERAL} \rangle$

(식별자) $id ::= \langle \text{IDENTIFIER} \rangle$

3 규칙 표기법

다음과 같은 표기법을 사용합니다:

$$\frac{\text{가정}_1 \quad \text{가설}_2 \quad \dots \text{가정}_n \quad \text{결론}}{\quad}$$

이는 $\text{가설}_1, \text{가설}_2, \dots, \text{가설}_n$ 을 모두 도출할 수 있다면, 결론 도 도출할 수 있다는 규칙입니다.

$n = 0$ 인 경우 특별한 경우가 발생하며, 이 경우를 다음과 같이 쓸 수 있습니다:

결론

아니면 가로줄을 생략하고 다음과 같이 쓸 수도 있습니다:

결론

이 경우를 가정이 없는 규칙이라고 할 수도 있고, *공리*라고 부를 수도 있습니다.

추론은 하나 이상의 공리를 출발점으로 삼아, 필요에 따라 몇 가지 규칙을 적용한 후, 최종적으로 결론에 도달하는 과정을 말합니다. 추론을 공리를 앞으로, 결론을 뿌리로 하는 나무 형태로 구성할 수 있다는 점에 유의하십시오. 이러한 나무를 *추론 나무*라고 부를 수 있습니다.

4 서브타이핑

이제 클래스 이름에 대해 *하위* 유형 관계를 정의합니다. 우리는 $\leq$ 를 사용하여 하위 유형 관계를 나타냅니다. $t_1 \leq t_2$ 일 때, t_1 이 t_2 의 하위 유형이라고 말합니다. 우리는 $\leq$ 가 반사적(reflexive)이고 전이적(transitive)이며, 클래스 간의 “extends” 관계에 의해 생성된다고 정의합니다.

$$t \leq t \tag{1}$$

$$\frac{t_1 \leq t_2 \quad t_2 \leq t_3}{t_1 \leq t_3} \tag{2}$$

$$\frac{\text{프로그램에 class } C \text{ extends } D \dots \text{가 포함되어 있습니다}}{C \leq D} \tag{3}$$

5 형 환경

형 환경은 식별자에서 형으로의 유한 매핑이다. 우리는 A 를 사용하여 형 환경을 표기한다. 우리는 $\text{dom}(A)$ 를 사용하여 A 의 정의역을 나타냅니다. 만약 $id_1, \dots, id_r$ 이 서로 다른 식별자라면, 표기법 $[id_1: t_1, \dots, id_r: t_r]$ 은 $i \in 1..r$ 에 대해 $id_i \mapsto t_i$ 로 매핑하는 유형 환경을 나타냅니다. A_1, A_2 가 타입 환경이라면, $A_1 \cdot A_2$ 는 다음과 같이 정의된 타입 환경이다:

$$(A_1 \cdot A_2)(id) = \begin{cases} A_2(id) & \text{if } id \in \text{dom}(A_2) \\ A_1(id) & \text{, 그 외의 경우} \end{cases} \tag{4}$$

$A_{(2)}$ 가 A_1 보다 우선한다는 점에 유의하십시오.

6 보조 함수

6.1 *classname* 헬퍼 함수

classname 함수는 클래스의 이름을 반환합니다. *classname*의 정의는 다음과 같습니다:

$$classname(class\ id\ \{ \text{public static void main (String [] } id^S \} \{ \dots \} \}) = id \quad (5)$$

$$classname(class\ id\ \{ t_1\ id_1 ; \dots ; t_f\ id_f ; m_1 \dots m_k \}) = id \quad (6)$$

$$classname(class\ id\ extends\ id^P\ \{ t_1\ id_1 ; \dots ; t_f\ id_f ; m_1 \dots m_k \}) = id \quad (7)$$

6.2 *linkset* 헬퍼 함수

linkset 함수는 클래스와 그 슈퍼클래스 간의 연결(한 쌍의 클래스 이름으로 구성된 싱글톤 집합으로 표현됨)을 반환하거나, 클래스에 슈퍼클래스가 없는 경우 빈 집합을 반환합니다. *linkset*의 정의는 다음과 같습니다:

$$linkset(class\ id\ \{ \text{public static void main (String [] } id^S \} \{ \dots \} \}) = \emptyset \quad (8)$$

$$linkset(class\ id\ \{ t_1\ id_1 ; \dots ; t_f\ id_f ; m_1 \dots m_k \}) = \emptyset \quad (9)$$

$$\text{링크셋(클래스 } id \text{ extends } id^P\ \{ t_1\ id_1 ; \dots ; t_f\ id_f ; m_1 \dots m_k \}) = \{(id, id^P)\} \quad (10)$$

6.3 *메서드명* 헬퍼 함수

methodname 함수는 메서드 정의의 이름을 반환합니다. *methodname*의 정의는 다음과 같습니다:

$$methodname(public\ t\ id^M\ (\dots) \dots) = id^M \quad (11)$$

6.4 *distinct* 헬퍼 함수

distinct 함수는 목록에 있는 식별자들이 서로 다른지 확인합니다.

$$\frac{\forall i \in 1..n : \forall j \in 1..n : id_i = id_j \Rightarrow i = j}{distinct(id_1, \dots, id_n)} \quad (12)$$

6.5 *비순환* 헬퍼 함수

비순환 함수는 쌍의 집합에 순환이 포함되어 있지 않은지 확인합니다.

$$\frac{\neg [\exists j_1, \dots, j_h \in 1..n : (\forall i \in 1..(h-1) : id^P = id_{j_i}) \wedge (id^P = id_{j_h})]}{\text{비순환}(\{(id_1, id^P), \dots, (id_n, id^P)\})} \quad (13)$$

6.6 *필드* 헬퍼 함수

우리는 *fields*(*C*)라는 표기법을 사용하여 *C*의 필드와 *C*의 슈퍼클래스들의 필드로 구성된 타입 환경을 나타냅니다. *C*의 필드는 *C*의 슈퍼클래스들의 필드보다 우선합니다. *fields*의 정의는 다음과 같습니다:

$$\frac{\text{class } id\ \{ t_1\ id_1 ; \dots ; t_f\ id_f ; m_1 \dots m_k \} \text{가 프로그램에 존재할 때}}{fields(id) = [id_1 : t_1, \dots, id_f : t_f]} \quad (14)$$

$$\frac{\text{class } id\ \text{extends } id^P\ \{ t_1\ id_1 ; \dots ; t_f\ id_f ; m_1 \dots m_k \} \text{가 프로그램에 포함되어 있다}}{fields(id) = fields(id^P) \cdot [id_1 : t_1, \dots, id_f : t_f]} \quad (15)$$

6.7 메서드 유형 헬퍼 함수

우리는 $methodtype(id, id^M)$ 표기법을 사용하여, 클래스 id (또는 id 의 슈퍼클래스) 내에서 이름이 id^M 인 메서드의 인자 유형 목록과 반환 유형(해당 메서드가 존재하지 않는 경우 $\perp$)을 함께 나타냅니다. $methodtype$ 의 결과는 $(t_1, \dots, t_n) \rightarrow t$ 또는 $\perp$ 의 형태를 띠다. $methodtype$ 의 정의는 다음과 같다:

$$\frac{\begin{array}{l} \text{class } id \{ \dots m_1 \dots m_k \} \text{가 프로그램에 존재하고, 어} \\ \text{떤 } j \in 1..k \text{에 대해 } methodname(m_j) = id^M \\ m_j \text{는 다음 형태를 가진다} \\ \text{public } t \text{ } id^M \text{ (} t_1^{id^F}, \dots, t_n^{id^F} \text{) } \{ t_1 id_1; \dots; t_r id_r; s_1 \dots s_q \text{ return } e; \} \end{array}}{methodtype(id, id^M) = (id_1^{id^F} : t_1^{id^F}, \dots, id_n^{id^F} : t_n^{id^F}) \rightarrow t} \quad (16)$$

class $id \{ \dots m_1 \dots m_k \}$ 가 프로그램에 존재할 때, 모

$$\frac{\begin{array}{l} \text{든 } j \in 1..k \text{에 대해: } methodname(m_j) \neq id^M \\ methodtype(id, id^M) = \perp \end{array}}{} \quad (17)$$

class id extends $id^P \{ \dots m_1 \dots m_k \}$ 가 프로그램에 존재하며, 어떤 $j \in 1..k$ 에 대해: $methodname(m_j) = id^M$

$$\frac{\begin{array}{l} m_j \text{는 다음 형태이다} \\ \text{public } t \text{ } id^M \text{ (} t_1^{id^F}, \dots, t_n^{id^F} \text{) } \{ t_1 id_1; \dots; t_r id_r; s_1 \dots s_q \text{ return } e; \} \end{array}}{methodtype(id, id^M) = (id_1^{id^F} : t_1^{id^F}, \dots, id_n^{id^F} : t_n^{id^F}) \rightarrow t} \quad (18)$$

class id extends $id^P \{ \dots m_1 \dots m_k \}$ 가 프로그램에 존재하며, 모든 j

$$\frac{\begin{array}{l} \in 1..k \text{에 대해: } methodname(m_j) \neq id^M \\ methodtype(id, id^M) = methodtype(id^P, id^M) \end{array}}{} \quad (19)$$

6.8 noOverloading 헬퍼 함수

$$\frac{methodtype(id^P, id^M) \neq \perp \Rightarrow methodtype(id^P, id^M) = methodtype(id, id^M)}{noOverloading(id, id^P, id^M)} \quad (20)$$

7 형식 규칙

7.1 유형 판단

다음과 같은 일곱 가지 유형의 판단을 사용할 것입니다:

- ▶ g
- ▶ mc
- ▶ d
- C ▶ m
- A, C ▶ s
- A, C ▶ $e : t$
- A, C ▶ $p : t$

판단을 다음과 같이 해석할 수 있습니다. 판단 ▶ g 는 “목표 g 의 타입 검사가 통과된다”를 의미합니다. 판단 ▶ mc 는 “메인 클래스 mc 의 타입 검사가 통과된다”를 의미합니다. 판단 ▶ d 는 “타입 선언 d 의 타입 검사가 통과된다”를 의미합니다. 판단 C ▶ m 은 “클래스 C 에서 정의된 경우, 메서드 선언 m 의 타입 검사가 통과된다”를 의미합니다. 판단 A, C ▶ s 는 “유형 환경 A 에서, 클래스 C 내에 작성된 경우 문장 s 의 유형이 유효하다”를 의미합니다. 판단 A, C ▶ $e : t$ 는 “유형 환경 A 에서, 클래스 C 내에 작성된 경우 표현식 e 의 유형이 t 이다”를 의미합니다. 판단 A, C ▶ $p : t$ 는 “유형 환경 A 에서, 클래스 C 내에 작성된 경우 주 표현식 p 의 유형이 t 이다”를 의미합니다.

7.2 목표

$$\begin{array}{l}
 \text{distinct}(\text{classname}(mc), \text{classname}(d_1), \dots, \text{classname}(d_n)) \\
 \text{acyclic}(\text{linkset}(d_1) \cup \dots \cup \text{linkset}(d_n)) \\
 \begin{array}{l}
 \text{▶ } mc \\
 \text{▶ } d_i \quad i \in 1..n
 \end{array}
 \end{array}
 \quad \frac{}{\text{▶ } mc \ d_1 \ \dots \ d_n} \quad (21)$$

7.3 메인 클래스

$$\frac{\text{distinct}(id_1, \dots, id_r) \quad [id_1 : t_1, \dots, id_r : t_r], \perp \text{▶ } s_i \quad i \in 1..q}{\text{▶ class } id \{ \text{public static void main (String [] } id^S \text{) } \{ t_1 \ id_1 ; \dots ; t_r \ id_r ; s_1 \ \dots \ s_q \} \}} \quad (22)$$

7.4 형식 선언

$$\frac{\begin{array}{l} \text{distinct}(id_1, \dots, id_f) \\ \text{distinct}(\text{methodname}(m_1), \dots, \text{methodname}(m_k)) \end{array} \quad \begin{array}{l} id \triangleright m_i \\ i \in 1..k \end{array}}{\triangleright \text{class } id \{ t_1 id_1 ; \dots ; t_f id_f ; m_1 \dots m_k \}} \quad (23)$$

$$\frac{\begin{array}{l} \text{distinct}(id_1, \dots, id_f) \\ \text{distinct}(\text{methodname}(m_1), \dots, \text{methodname}(m_k)) \\ \text{noOverloading}(id, id^p, \text{methodname}(m_i)) \end{array} \quad \begin{array}{l} id \triangleright m_i \\ i \in 1..k \end{array}}{\triangleright \text{class } id \text{ extends } id^p \{ t_1 id_1 ; \dots ; t_f id_f ; m_1 \dots m_k \}} \quad (24)$$

7.5 메서드 선언

$$\frac{\begin{array}{l} \text{distinct}(id^F, \dots, id^F, id_1, \dots, id_r) \\ A = \text{fields}(C) \cdot [id^F : t_1^F, \dots, id^F : t_r^F, id_1 : t_1, \dots, id_r : t_r] \\ A, C \triangleright s_i \quad i \in 1..q \quad A, C \triangleright e : t \end{array}}{C \triangleright \text{public } t id^M (t_1^F id_1^F, \dots, t_r^F id_r^F) \{ t_1 id_1 ; \dots ; t_r id_r ; s_1 \dots s_q \text{ return } e; \}} \quad (25)$$

7.6 문

$$\frac{A, C \triangleright s_i \quad i \in 1..q}{A, C \triangleright \{ s_1 \dots s_q \}} \quad (26)$$

$$\frac{A(id) = t_1 \quad A, C \triangleright e : t_2 \quad t_2 \leq t_1}{A, C \triangleright id = e; } \quad (27)$$

$$\frac{A(id) = \text{int[]} \quad A, C \triangleright e_1 : \text{int} \quad A, C \triangleright e_2 : \text{int}}{A, C \triangleright id [e_1] = e_2 ; } \quad (28)$$

$$\frac{A, C \triangleright e : \text{부울} \quad A, C \triangleright s_1 \quad A, C \triangleright s_2}{A, C \triangleright \text{if } (e) s_1 \text{ else } s_2} \quad (29)$$

$$\frac{A, C \triangleright e : \text{부울} \quad A, C \triangleright s}{A, C \triangleright \text{while } (e) s} \quad (30)$$

$$\frac{A, C \triangleright e : \text{int}}{A, C \triangleright \text{System.out.println}(e); } \quad (31)$$

7.7 식 및 기본식

$$\frac{A, C \triangleright p_1 : \text{부울} \quad A, C \triangleright p_2 : \text{부울}}{A, C \triangleright p_1 \ \&\& \ p_2 : \text{부울}} \quad (32)$$

$$\frac{A, C \triangleright p_1 : \text{정수} \quad A, C \triangleright p_2 : \text{정수}}{A, C \triangleright p_1 < p_2 : \text{부울}} \quad (33)$$

$$\frac{A, C \triangleright p_1 : \text{정수} \quad A, C \triangleright p_2 : \text{정수}}{A, C \triangleright p_1 + p_2 : \text{int}} \quad (34)$$

$$\frac{A, C \triangleright p_1 : \text{int} \quad A, C \triangleright p_2 : \text{int}}{A, C \triangleright p_1 - p_2 : \text{int}} \quad (35)$$

$$\frac{A, C \triangleright p_1 : \text{int} \quad A, C \triangleright p_2 : \text{int}}{A, C \triangleright p_1 * p_2 : \text{int}} \quad (36)$$

$$\frac{A, C \triangleright p_1 : \text{int}[] \quad A, C \triangleright p_2 : \text{int}}{A, C \triangleright p_1 [p_2] : \text{int}} \quad (37)$$

$$\frac{\begin{array}{c} A, C \triangleright p : \text{int}[] \\ A, C \triangleright p.\text{length} : \text{int} \\ A, C \triangleright p : D \quad \text{methodtype}(D, id) = (t'_1, \dots, t'_n) \rightarrow t \\ A, C \triangleright e_i : t_i \quad t_i \leq t'_i \quad i \in 1..n \end{array}}{A, C \triangleright p.id(e_1, \dots, e_n) : t} \quad (38)$$

$$A, C \triangleright c : \text{int} \quad (40)$$

$$A, C \triangleright \text{true} : \text{부울} \quad (41)$$

$$A, C \triangleright \text{false} : \text{부울} \quad (42)$$

$$\frac{id \in \text{dom}(A)}{A, C \triangleright id : A(id)} \quad (43)$$

$$\frac{C \neq \perp}{A, C \triangleright \text{this} : C} \quad (44)$$

$$\frac{A, C \triangleright e : \text{int}}{A, C \triangleright \text{new int}[e] : \text{int}[]} \quad (45)$$

$$A, C \triangleright \text{new id}() : id \quad (46)$$

$$\frac{A, C \triangleright e : \text{부울}}{A, C \triangleright !e : \text{부울}} \quad (47)$$

$$\frac{A, C \triangleright e : t \quad A, C \triangleright (e) : t}{A, C \triangleright e : t} \quad (48)$$